

→ Milestone 7-8

> the core mental

PyTorch Model

yoloV8n.pt
|
trainable / framework
code
|
PyTorch specific

ONNX Model

yoloV8n.onnx
|
Portable graph
recipe
|
runtime
independent

framework
independent
computation
graph

TensorRT engine

yoloV8n.engine
|
compiled GPU
executable
|
opt. for 24

- **PyTorch model:** the original recipe written in PyTorch's language.
- **ONNX:** a universal recipe describing operations and tensors.
- **TensorRT:** an NVIDIA chef who studies that recipe, tries different cooking methods, and creates an optimized kitchen plan for one GPU.

→ ONNX

Suppose:

```
def forward(x):  
    y = x * 2  
    return y + 1
```

> PyTorch executes Python and framework ops!

Python func. → Multiply by 2 → add 1 → output

(→ when it's exported to ONNX
it becomes graph

input x → Mul($x, 2$) → Add(result, 1) → output y

> Computation Graph

collection of ops. connected by tensors:

img tensor $\rightarrow$ convolution $\rightarrow$ activation $\rightarrow$ another convolution $\rightarrow$ prediction tensor

each op is Node

each connection carries a tensor

$\hookrightarrow$ flowchart where every box performs math and every arrow carries numbers

We provide the recipe $\rightarrow$ ONNX
TensorRT designs the fast kitchen $\rightarrow$ engine
We write code to operate it $\rightarrow$ TensorRT backend

instead of
writing new
model to TensorRT
feed ONNX to it.
file

> ONNX model contains:

- inputs $\rightarrow$ name, shape, dtype
- nodes $\rightarrow$ represents one computation (conv, add, mul, sigmoid, concat, resize, transpose)
- tensors $\rightarrow$ data flowing
- initializers $\rightarrow$ trained parameters (convolution weights, bias values, constant tensors)
- outputs

$\hookrightarrow$ Input tensor = ingredients brought by the customer
Initializers = equipment already installed in the kitchen

> Non Maximum Suppression (NMS) $\hookrightarrow$ Yolo returns 8400 detection
NMS cleans up to
 ~ 6 detections

cleanup step

the model may produce several overlapping boxes around the same 'bus'
keeps the strongest box and removes highly overlapping weaker copies

! opset: versioned collection of ONNX operator definitions

intended graph contract

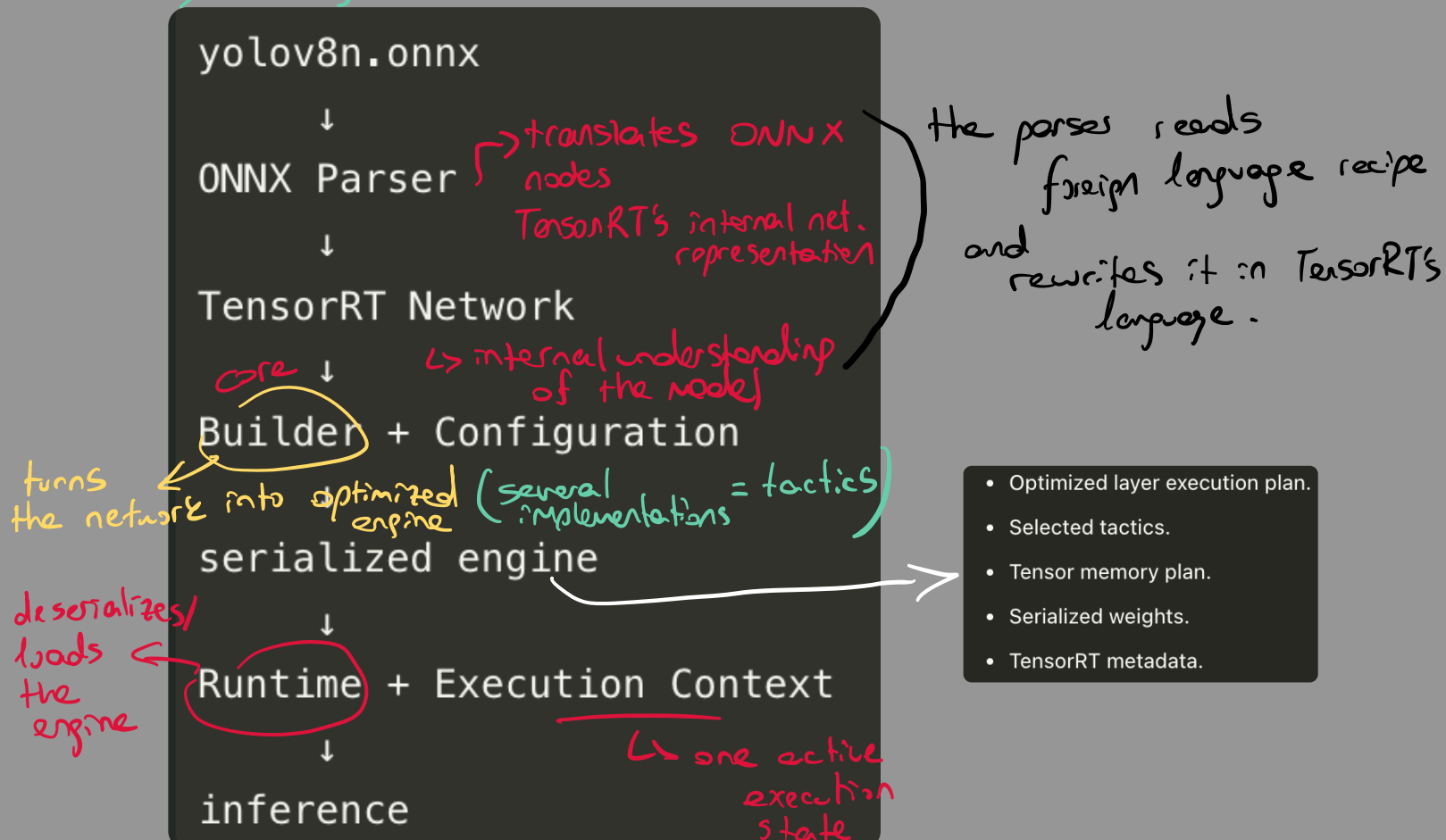
Input:
images
FP32 [1,3,640,640]

Graph:
YOLOv8n model operations and trained weights

Output:
raw predictions
approximately [1,84,8400]

Outside graph:
decoding
letterboxing
CUDA preprocessing
NMS
drawing

→ feeding into TensorRT



⇒ TensorRT writes predictions directly into memory
it does not return a new tensor like a normal func.

↳ must allocate output Tensor in advance

engine file → runtime opens it → live engine → context operates it
blueprint → loader → machine → machine operator

1. ONNX created a portable description of YOLO's computation.
2. We inspected its nodes, shapes and types.
3. We verified ONNX and PyTorch produced equivalent outputs.
4. TensorRT compiled ONNX into an L4-specific engine.
5. Our runtime:
 - Deserialized the engine
 - Created an execution context
 - Allocated input/output GPU memory
 - Bound CUDA addresses
 - Launched inference
6. We aligned TensorRT and PyTorch precision settings and passed correctness:
 - Mean difference: 0.00000248
 - allclose: True

> next is fp16

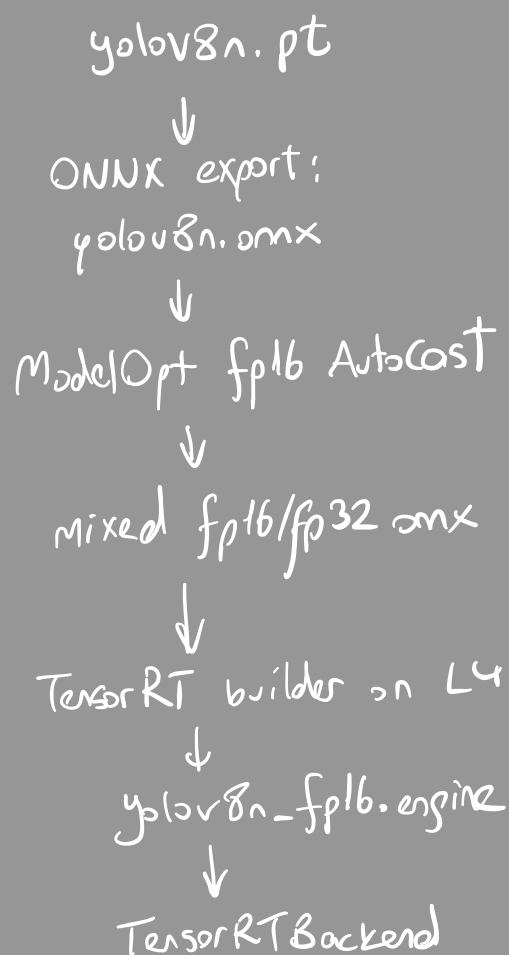
fp32 onnx
↓ ModelOpt AutoCast
mixed fp16/fp32 onnx
↓ existing TensorRT builder
fp16 optimized engine

↳ decides fp16/fp32 placement

↓
onnx records types

↓
builder obey them

Scope	PyTorch FP32	TensorRT FP16	Speedup
Model only	9.529 ms	1.428 ms	6.671x
Complete pipeline	13.600 ms	9.664 ms	1.407x



↳ raw model benchmark

↳ Ultralytics preprocessing/NMS/visualization benchmark

! **model-only**: uses CUDA events because it measures GPU execution
 • **end-to-end**: uses synchronized CPU wall clock because it includes CPU and GPU work:

decode → preprocessing → transfer → model

For bus.jpg :

1. OpenCV decodes the file into a BGR HWC array.
2. Ultralytics resizes and letterboxes it to 640 × 640 .

visualization ← NMS ←

3. Data becomes normalized FP16 BCHW.
4. TensorRT receives `[1, 3, 640, 640]`.
5. The engine produces `[1, 84, 8400]` raw candidates.
6. Confidence filtering and NMS remove overlapping/weak candidates.
7. Five final detections remain.
8. `result.plot()` renders those detections.

Errors are caught at each boundary:

- Missing engine → `FileNotFoundError`.
- CPU input → `ValueError`.
- Incorrect shape/dtype → `ValueError`.
- Unsupported dynamic shape → `ValueError`.
- Failed TensorRT launch → `RuntimeError`.
- Incorrect final detections → correctness-gate failure.

! Amdahl's Law!
$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

> predicts the maximum theoretical speedup of a task when using multiple processors
OR
parallel computing.

1. Conceptual: What is the difference between an ONNX graph and a TensorRT engine?
2. Conceptual: Why did the model-only speedup shrink in the complete pipeline?
3. Code tracing: Which objects are created once in `TensorRTBackend.__init__()`, and which actions happen in `infer()`?
4. Code tracing: How can raw FP16 `allclose` fail while the final detection gate passes?
5. Design: What would need to change to support dynamic image shapes or batch sizes?

1. onnx graph stores the the model framework-independent
TensorRT's builder recipe and produces an already optimized engine
2. CPU/stored stages are included on end-to-end measurements
3. deserialize engine
create exec. context ;
alloc the output
bind the input tensor's address
execute TRT
4. some of the candidates may exceed tolerance
after filtering and NMS those candidates
are removed

5. static:

one engine shape $\rightarrow [1, 3, 640, 640]$

dynamic:

supported range $\rightarrow$ min / optima / max shapes